Tech365

Leveraging technology for growth

TERRAFORM

Infrastructure as Code with AWS

Build • Automate • Manage Cloud Infrastructure

Providers • State • Modules • CI/CD

What Is Cloud Infrastructure?

Tech365
Leveraging technology for growth

The computing resources that applications need to run, store data, and communicate over the internet.

Compute

Runs your application

Example:
Amazon EC2

Storage

Keeps files, images, and backups

Example:
Amazon S3

Networking

Connects resources and controls traffic

Example:
Amazon VPC

Databases

Stores organised application data

Example:
Amazon RDS

Security & Access

Controls who can access resources

Example:
AWS IAM

Example: An online store needs servers, product-image storage, a database, networking, and access controls.

Manual Provisioning vs. Provisioning Through Code

Tech365
Leveraging technology for growth

Provisioning means creating and configuring infrastructure.

Manual Provisioning

- Create resources by clicking in a cloud console.
- Repeat the setup steps for each environment.
- Record changes separately.
- Useful for learning and small, one-off tasks.

Example: Create a server, network, and database one at a time in AWS.

Provisioning Through Code

- Describe resources in configuration files.
- Reuse the configuration across environments.
- Review and track changes in Git.
- Useful for repeatable deployments and team workflows.

Example: Define the resources in Terraform, review the plan, then apply it.

Both create real infrastructure. Code makes the setup easier to repeat, review, and maintain.

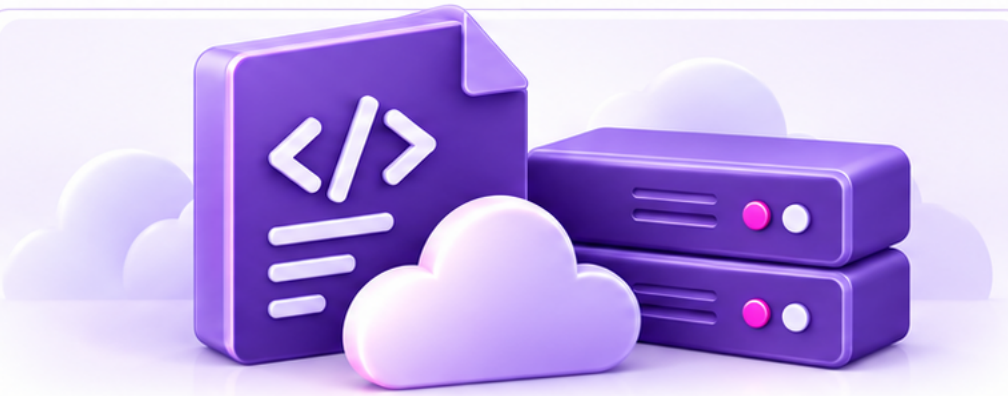
What Is Infrastructure as Code?

Tech365
Leveraging technology for growth

Infrastructure as Code (IaC) means defining and managing infrastructure using configuration files that tools can read and apply.

Your infrastructure becomes a written specification

- Describe the servers, networks, and storage you need.
- Use a tool such as Terraform to create or update them.
- Save the files in Git to review and track changes.



Simple example

Define two web servers and a network in Terraform files. Review the plan, then apply it to provision the resources.

Repeatable setups

Reviewable changes

Reusable configurations

The code describes the infrastructure. The tool makes the changes.

Problems Terraform Helps Solve

Tech365
Leveraging technology for growth

Manage infrastructure with repeatable, reviewable configuration.

Common problem	How Terraform helps
Repetitive manual setup	Reuse code to create resources without repeating console steps.
Inconsistent environments	Use shared modules and explicit settings for consistent setups.
Unclear infrastructure changes	Review a plan before applying changes; track code in Git.
Changes made outside the code	Detect differences during planning and decide how to resolve them.
Complex resource dependencies	Use resource references to determine the order of operations.

Terraform reduces repetitive work and makes infrastructure changes easier to review.

Correct code, access controls, testing, and backups are still needed.

Terraform
Providers & Supported Platforms

Tech365
Leveraging technology for growth

A provider is a plugin that lets Terraform communicate with a platform through its API to manage resources.

Amazon Web Services
Provider: hashicorp/aws

EC2 instances, S3 buckets, VPCs

Microsoft Azure
Provider: hashicorp/azurerm

Virtual machines, networks, storage

Google Cloud
Provider: hashicorp/google

Compute instances, storage, networks

Kubernetes
Provider: hashicorp/kubernetes

Deployments, services, namespaces

GitHub
Provider: integrations/github

Repositories, teams, branch protection

Cloudflare
Provider: cloudflare/cloudflare

DNS records and supported security settings

Find providers in the Terraform Registry. Each provider supports specific resources and data sources.

Providers may be maintained by HashiCorp, partners, or the community.

Terraform, Docker, Kubernetes & Ansible

Tech365
Leveraging technology for growth

Different tools handle different parts of application delivery.

Terraform

Provision infrastructure

Creates and manages cloud resources such as networks, servers, and clusters.

Example: Provision an AWS VPC and an EKS cluster.

Docker

Package and run containers

Builds application images and runs containers from those images.

Example: Package a Python application and its dependencies.

Kubernetes

Manage container workloads

Schedules containers, replaces failed workloads, and manages scaling.

Example: Run three application replicas across a cluster.

Ansible

Configure systems and automate tasks

Installs software, manages configuration, and automates server tasks.

Example: Configure Nginx and application services on EC2.

Working together

Terraform creates infrastructure. Ansible configures servers where needed. Docker builds images. Kubernetes runs and manages the workloads.

Not every project needs all four. Their capabilities can overlap.

Configuration, Infrastructure & State

Tech365
Leveraging technology for growth

Three different things Terraform uses to manage resources.

Configuration

What do you want?

Your .tf files describe the resources and settings you want Terraform to manage.

Example: An EC2 instance tagged Name = web.

Real Infrastructure

What exists now?

The actual resources running in AWS or another platform.

Example: The EC2 instance currently running in your AWS account.

Terraform State

What does Terraform track?

A record linking resource addresses in your code to real resource IDs, with last-known attributes.

Example: aws_instance.web is linked to its EC2 instance ID.

What happens during terraform plan?

Terraform normally refreshes tracked resources through the provider, compares them with your configuration, and proposes the changes needed.

Configuration is your intent. Infrastructure is reality. State is the mapping.

State is not a backup of your application d ata. It can contain secrets and must be protected.

Installing Terraform

Tech365
Leveraging technology for growth

Install the command-line tool on your computer.

Windows
Download the binary

1. Download the Windows ZIP for your CPU.
2. Extract terraform.exe to C:\Terraform.
3. Search for “Edit environment variables for your account”.
4. Edit Path and add C:\Terraform.
5. Reopen PowerShell.

macOS
Using Homebrew

With Homebrew installed, run:

```
brew tap hashicorp/tap
```

```
brew install hashicorp/tap/terraform
```

Reopen your terminal if needed.

Linux
Manual installation

1. Download the Linux ZIP for your CPU.
2. Extract the archive.
3. From the extracted folder, run:

```
sudo install -m 0755 terraform
/usr/local/bin/terraform
```

Ensure /usr/local/bin is in your PATH.

Verify your installation

```
terraform -version
```

A Terraform version should appear.
If not, check PATH and reopen your terminal.

Official downloads: developer.hashicorp.com/terraform/install

Installing & Configuring the AWS CLI

Tech365
Leveraging technology for growth

Use terminal commands to interact with your AWS account.

1 Install AWS CLI v2

Download the installer for your operating system from **aws.amazon.com/cli/**

Windows:
Run the MSI installer.

macOS:
Run the PKG installer.

Linux:
Use the official ZIP installation steps.

Reopen your terminal and check:

```
aws --version
```

2 Configure a profile

With IAM Identity Center (SSO)

Ask your administrator for the start URL and SSO region.

```
aws configure sso --profile training
```

Follow the browser sign-in. Select your account and role, then set a default AWS region and JSON output.

To sign in again:

```
aws sso login --profile training
```

3 Verify your identity

Run this command on one line:

```
aws sts get-caller-identity --profile training
```

Confirm the returned Account and Arn match the account and role you intend to use.

This verifies sign-in, not permission to create every resource.

Using lab access keys instead?

Run `aws configure --profile training` and enter your IAM access key, secret key, region, and output format.

Never use root access keys or put credentials in Terraform files or Git.

Authenticate to AWS Without Credentials in Code

Tech365
Leveraging technology for growth

Terraform can use a local AWS profile or an IAM role.

On your computer: use an SSO profile

1 Configure once (IAM Identity Center required)

```
aws configure sso --profile training
```

2 Sign in

```
aws sso login --profile training
```

3 Select the profile in your terminal

Windows PowerShell:

```
$env:AWS_PROFILE = "training"
```

macOS / Linux:

```
export AWS_PROFILE=training
```

4 Verify the selected identity

```
aws sts get-caller-identity
```

Keep the Terraform provider simple

```
provider "aws" {
  region = "us-east-1"
}
```

Example region: choose the AWS region you need.

The AWS provider uses the selected profile.
No access key or secret key belongs in this file.

For automated runs

- On EC2: use an attached IAM role.
- In CI/CD: use OIDC federation to assume an IAM role.

These methods provide temporary credentials.

Keep credentials outside .tf files, .tfvars files, and Git.
Use a clean terminal without old AWS access-key environment variables, which can override profile credentials.

Configuring the AWS Provider & Region

Tech365
Leveraging technology for growth

Tell Terraform which provider to use and where to create regional resources.

Create main.tf

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Understand the configuration

`required_providers`	Declares the providers your project needs.
`source = "hashicorp/aws"`	Identifies the AWS provider in the Terraform Registry.
`region = "us-east-1"`	Sets the default region for resources using this provider.

Example regions

```
us-east-1: US East (N. Virginia)
eu-west-1: Europe (Ireland)
```

Initialise and validate

`terraform init`

Downloads the provider and initialises the project.

`terraform validate`

Checks configuration syntax and internal consistency.

Use your configured AWS profile or IAM role. Keep credentials out of the code.
This minimal example omits version constraints; add them for shared projects.

Creating Your First Resource

Example: Create an Amazon S3 bucket with Terraform.

Tech365
Leveraging technology for growth

1. Add this to main.tf

Keep the AWS provider configuration from the previous slide.

```
resource "aws_s3_bucket" "demo" {
  bucket_prefix = "tech365-demo-"

  tags = {
    Name        = "Terraform Practice"
    Environment = "Learning"
  }
}
```

`aws_s3_bucket:`	Resource type
`demo:`	Local name used in Terraform
`bucket_prefix:`	Terraform adds a unique suffix

2. Run in your project folder

1	`terraform init`	Initialise the project and install the provider.
2	`terraform fmt`	Format your configuration.
3	`terraform validate`	Check the configuration.
4	`terraform plan`	Review the proposed bucket creation.
5	`terraform apply`	Review the plan again, then type yes to approve.

3. Verify the result

Open Amazon S3 in the AWS Console and find the new bucket. To inspect its tracked details, run:

```
terraform state show aws_s3_bucket.demo
```

Clean up after practice: `terraform destroy.` The bucket must be empty before deletion.

Understanding .terraform & .terraform.lock.hcl

Terraform manages both during initialisation, but they serve different purposes.

Tech365
Leveraging technology for growth

.terraform/

Local working directory.

1. Stores installed providers and downloaded modules.
2. Stores backend and workspace metadata.
3. Created or updated by `terraform init`.
4. Keep it out of Git.

Add to .gitignore

```
.terraform/
```

.terraform.lock.hcl

Provider dependency lock file.

1. Records selected provider versions and package checksums.
2. Helps teammates install the same provider versions.
3. Managed automatically by Terraform.
4. Commit it to Git with your configuration.

It **does not lock** module versions or the Terraform CLI version.

`terraform init`

Reuses locked provider versions when they satisfy your constraints.

`terraform init -upgrade`

Selects newer allowed versions. Review and test the changes.

Ignore the local folder. Keep the provider lock file.

The dependency lock file is different from a state lock and from terraform.tfstate.

Which Terraform Files Belong in Git?

Share the configuration. Protect secrets and generatded files.

Tech365
Leveraging technology for growth

COMMIT

Files that should be version controlled in Git.

1. ***.tf files**
   Infrastructure code without secrets.
2. **.terraform.lock.hcl**
   Provider versions and checksums.
3. **Local module source and tests**
   Reusable code and validation.
4. **README.md and .gitignore**
   Instructions and repository rules.
5. **terraform.tfvars.example**
   Safe placeholders only.

IGNORE

Files that should not be committed to Git.

1. **.terraform/**
   Downloaded providers and local metadata.
2. ***.tfstate and backups**
   State can contain sensitive data.
3. **Saved plan files**
   Plans can contain secrets.
4. **Credential files and .env**
   Keep authentication details private.
5. **Secret variable files and crash logs**
   Never commit sensitive values.

Example .gitignore

Add these entries to your .gitignore file.

```
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
tfplan
.env
.env.*
*.tfvars
*.tfvars.json
crash.log
crash.*.log
```

Use .tfplan for saved plans, or ignore the exact filename you choose.

What about .tfvars?

This example ignores all variable-value files. Non-secret environment settings may be committed deliberately after review.

Commit .terraform.lock.hcl. Do not ignore it.

Gitignore does not remove files already tracked by Git.

Introduction to HCL

Tech365
Leveraging technology for growth

HCL stands for HashiCorp Configuration *Language*.

Terraform uses HCL syntax in `.tf` files to describe the infrastructure you want.

1 A simple Terraform configuration

```
resource "aws_s3_bucket" "demo" {
  bucket_prefix = "tech365-demo-"

  tags = {
    Environment = "Learning"
  }
}
```

This declares an S3 bucket with a name prefix and a tag.

2 Read the building blocks

1	**Block type:** resource	Declares an infrastructure resource.
2	**Labels:** "aws_s3_bucket" and "demo"	Specify the resource type and its local name.
3	**Argument:** bucket_prefix = ...	Assigns a value to a setting.
4	**Braces:** { }	Enclose block bodies or object values.
5	**String:** "Learning"	Text values use double quotes.

1 Arguments use name = value
Set configuration values with an equals sign.

2 Comments can start with #
Use # for single-line comments.

3 tags = { ... } is an object value
The tags argument takes an object value with key-value pairs.

Terraform configuration is declarative: describe the desired result, and Terraform plans the changes.

Input Variables & Default Values

Tech365
Leveraging technology for growth

Make your Terraform configuration reusable by passing in values.

1 Declare in variables.tf

```
variable "environment" {
  description = "Deployment environment"
  type        = string
  default     = "dev"
}
```

type: defines the expected value type.

default: used when no value is supplied.

description: explains the input.

2 Use in main.tf

```
resource "aws_s3_bucket" "demo" {
  bucket_prefix = "tech365-demo-"
  tags = {
    Environment = var.environment
  }
}
```

Reference the variable with **var.environment**.

Do not put this reference inside quotes.

3 Override the default

In terraform.tfvars:

```
environment = "staging"
```

Or for a single command:

```
terraform plan -var="environment=prod"
```

The command-line value takes priority over terraform.tfvars and the default.

Which value is used?

No supplied value:	With the .tfvars value:	With the command above:
dev	**staging**	**prod**

Without a default, a variable is required. Terraform asks for it interactively or errors when input is disabled.

Data Sources: Read Existing Information

Tech365
Leveraging technology for growth

A data block reads information through a provider without managing the source resource.

1 Example: Look up an existing VPC

Add this to **main.tf** with your AWS provider configured.

```
data "aws_vpc" "existing" {
  id = "vpc-0123456789abcdef0"
}

output "vpc_cidr" {
  value = data.aws_vpc.existing.cidr_block
}
```

Replace the example ID with a real VPC ID in your AWS account and configured region.

2 Understand the example

data	Declares a read operation.
aws_vpc	The AWS data source type.
existing	The local name used in this configuration.
data.aws_vpc.existing.cidr_block	References the CIDR range returned by the lookup.

resource: Creates and manages a resource.

data: Reads information about an existing resource.

Try it

1. `terraform init` | 2. `terraform plan` | 3. `terraform apply`

Review the lookup and output, then approve to save the output in state.

Additional command:

`terraform output vpc_cidr`

Display the saved CIDR value after apply.

Reading a VPC does not import it or make Terraform manage its lifecycle.

Organising Terraform Files

Separate files by purpose to make your project easier to understand.

Tech365
Leveraging technology for growth

File or folder	Purpose
versions.tf	Terraform version requirements and required providers.
providers.tf	AWS provider settings, such as the region.
main.tf	Resources, data sources, and module calls.
variables.tf	Input variable declarations and defaults.
outputs.tf	Values to display or expose to other configurations.
terraform.tfvars	Values for declared variables. Review before sharing.
modules/	Reusable local modules, each in its own subfolder.
README.md and .gitignore	Project instructions and files Git should ignore.

These filenames are conventions
Terraform reads all .tf files in the current module directory together. File names do not control execution order.

Subfolders are separate modules.
Terraform does not automatically load their .tf files into the parent module.

Commit .terraform.lock.hcl. Ignore .terraform/, state files, saved plans, and secrets.

Provisioning AWS Networking

Use Terraform to define the network your applications run in.

Tech365
Leveraging technology for growth

VPC
Your isolated network in AWS.

Terraform:
`aws_vpc`

Subnets
Divide the VPC into sections, each in one Availability Zone.

Terraform:
`aws_subnet`

Internet Gateway
Enables internet connectivity when routes and resource settings allow it.

Terraform:
`aws_internet_gateway`

Route Tables
Choose where traffic goes; associate them with subnets.

Terraform:
`aws_route_table`
`aws_route_table_association`

NAT Gateway
Lets private IPv4 workloads initiate internet connections through a public subnet.

Terraform:
`aws_nat_gateway`

Security Groups
Control allowed traffic to and from resources.

Terraform:
`aws_security_group`

Public or private?

Public subnet: its route table has a route to an internet gateway.
Private subnet: no direct route to an internet gateway; internet egress can use NAT.

IPv4 internet access also needs suitable IP addressing and security rules. NAT gateways inncur charges.

Provisioning EC2 for Applications

Create the server, connect it to your network, and prepare it to run your application.

Tech365
Leveraging technology for growth

Example resource in main.tf

```
resource "aws_instance" "app" {
  ami                    = var.ami_id
  instance_type          = "t3.micro"
  subnet_id              = var.subnet_id
  vpc_security_group_ids = [var.app_sg_id]

  metadata_options {
    http_tokens = "required"
  }

  tags = {
    Name = "app-server"
  }
}
```

Declare and supply ami_id, subnet_id, and app_sg_id.
Configure the AWS provider first.

What the instance needs

1 Machine image (AMI)
Choose an image in your region compatible with the instance type.

2 Compute size
Select CPU and memory for the application.
t3.micro is a lab example.

3 Network and security
Use a subnet and security group in the same VPC.
Allow only required traffic.

4 Application setup
Install the runtime, deploy the code, and run it as a service.

5 Secure administration
Use Session Manager with the required IAM role, agent, and connectivity, or restrict SSH to your IP.

Deploy and verify

`terraform init` | `terraform plan` | `terraform apply`

Check EC2 status checks, application health, and logs after deployment.

Key takeaway

Creating an EC2 instance does not automatically install your application.

Use user data, Ansible, a prepared AMI, or a deployment pipeline for setup. EC2 usage can incur charges.